Joules Per Task: Energy Measurements for Learned Policies on Embedded Hardware, and an Energy-Accuracy Analysis for a Vision-Language-Action Policy

Emilio Girard Independent researcher, Montréal, Canada. emiliogirard.com

Correction, 2026-11-09. The first version of this paper paired the energy of the R+I configuration at τ=0.05 (2.124 J per step, 71.7% simulator skip rate) with a success rate of 68%. The 100-episode simulator evaluation at τ=0.05 scored 51%; the 68% belongs to the τ=0.02 evaluation, which had a 34.6% skip rate. Corrected, R+I at τ=0.05 costs 2.124 x 15,650 / 51, about 652 J per successful task (32% below the 963 J baseline), at 51% success. The drop from 70% to 51% is significant (two-proportion z=2.75), so the earlier claims of 489 J, a 49% reduction and success indistinguishable from baseline are withdrawn. The 652 J figure also understates the true cost, because it uses the baseline step count and failed episodes run longer, and because the energy was measured at an 82.4% replay skip rate against 71.7% in the simulator. R+I at τ=0.02 was not measured. R alone at τ=0.02 costs 3.415 J per step, about 786 J per successful task at 68% success (18% below baseline, success not significantly different). Other changes: the R-only τ=0.05 row now shows 51% success and its measured 2.811 J per step; the failure boundary reads 51% to 4% between 72% and 93% skip; the claim that one-step policies are more sensitive to visual staleness is withdrawn because the corrected data do not show it (19 points lost by the ten-step policy against 15 by the one-step policy at τ=0.05); the 100 Wh battery figure is 374 successful tasks, not 274; synthetic inputs overstate baseline energy by 37%, not 27%; the baseline latency row now uses the 232.9 ms measured on the same recorded frames as the 4.310 J figure; Figures 1 and 3 were redrawn. The 963 J baseline and the four-policy spectrum are unchanged.

Abstract

Although energy availability constrains the mission duration of mobile robots, the energy consumption of vision-language-action (VLA) policies is rarely reported: a recent survey of VLA efficiency metrics identifies direct energy estimation as an open problem and adopts kinematic proxies in its place [1], and of the VLA inference-efficiency works reviewed here only a concurrent cross-accelerator characterization [15] reports measured energy, on Jetson AGX Orin and Thor, as a column separate from task success. This paper presents hardware energy measurements for a VLA policy on the smallest Jetson module, normalized by task success, evaluated against a standard manipulation benchmark. SmolVLA (450M parameters) [2] is deployed on a Jetson Orin Nano through a multi-engine TensorRT decomposition whose outputs agree with fp32 reference execution to a maximum action-space deviation of 2.5e-3; power is sampled from the module's INA3221 instrumentation at 10 ms intervals, and task success is evaluated on LIBERO-Spatial [3] with 100 episodes per configuration. The baseline configuration consumes 963 J (0.27 Wh) per successful task at 70% success. Three inference optimizations are then evaluated jointly for energy and success: (i) threshold-gated reuse of visual encodings across control steps; (ii) an incremental prefill that exploits the structure of the policy's prefix attention mask to update a single token exactly, in place of full 177-token recomputation; and (iii) one-step distillation of the flow-matching action head under a low-rank

adaptation constraint. Visual reuse alone at a threshold of 0.02 reduces energy to about 786 J per successful task, an 18% reduction, at 68% success against 70% for baseline (n=100 per condition, z=0.31, not significant). Adding the exact incremental prefill at the more aggressive threshold of 0.05 lowers energy to about 652 J per successful task, 32% below baseline, but success falls to 51% (z=2.75 against baseline) and the figure understates the true energy cost because failed episodes run longer than the baseline step count assumes; the combination at the threshold of 0.02 was not measured. Neither requires additional training. The full combination reaches 0.68 J per control step at 21 Hz, a 6.3-fold energy reduction and 4.9-fold latency reduction, as an energy floor outside the usable operating range. The analysis further establishes the failure boundary of visual reuse (success falls from 51% to 4% between 72% and 93% skip rates, against 68% at 35%), finds no evidence that one-step distilled policies are more sensitive to stale visual conditioning than their ten-step teachers (15 versus 19 points of success lost at the threshold of 0.05), and documents six deployment failure modes, including silent corruption of prefill activations caused by fp16 normalization overflow under TensorRT. Three further policies spanning 0.2M to 450M parameters are then measured on the same device and rail, establishing that parameter count does not order energy cost: a 19M-parameter diffusion policy consumes 5.1 times the energy of a 30M-parameter transformer policy because it executes ten sequential denoising passes per control step. Applying the visual-reuse gate to both, beyond the failure boundary and therefore as an energy floor rather than a deployable configuration, returns 18.9x on the transformer policy and 1.55x on the diffusion policy, showing that the payoff of an inference optimization is predictable from where a policy's sequential computation resides. All measurements are reproducible from released scripts.

1. Introduction

Evaluations of robot manipulation policies report task success and, increasingly, control latency. Energy consumption is generally absent, despite the fact that onboard computation and actuation draw from a shared battery budget and that battery capacity improves slowly relative to the growth in model size. The omission appears to be practical rather than principled. Credible energy measurement requires simultaneously (a) executing a modern VLA on power-instrumented embedded hardware, for which mainstream robot-learning frameworks provide no runtime, and (b) validating task success, which requires a benchmark environment. Li et al. [1] state that reliable energy estimation is not achievable in simulation and substitute kinematic proxies such as jerk and path length. Vendor materials for embedded platforms report latency and relative chip-level efficiency figures, but not absolute energy for policy workloads. The one exception, a cross-accelerator study posted while this work was in progress [15], measures energy on Jetson AGX Orin and Thor and treats it as a platform property rather than as a quantity to be traded against accuracy per optimization.

This paper makes five contributions:

1. A metric and its instantiation. Joules-per-successful-task is defined as measured module energy divided by the number of benchmark successes, with success evaluated in the standard simulator and energy integrated from power-rail measurements on hardware executing a numerically validated implementation of the identical policy.
2. A deployment pipeline for SmolVLA-class policies on Orin-class devices, for which no PyTorch runtime exists. The pipeline decomposes inference into separately built TensorRT engines with

an exact, stateless per-step key-value cache contract; its construction surfaced six failure modes that are documented for reproducibility (Section 3.1).
3. A joint energy-accuracy analysis of three inference optimizations, spanning exact (no accuracy risk) to lossy (measured accuracy cost), including the failure boundary of the most aggressive setting.
4. A four-policy energy comparison spanning three orders of magnitude of parameter count on a single device and rail, establishing that sequential pass count rather than parameter count governs edge energy, and that the return on a given inference optimization is predictable from the policy's structure (Section 6.4).
5. Two empirical observations with design implications: first, about a third of visual encoder invocations in the benchmark workload (34.6% skipped) can be elided without measurable effect on success, although vision accounts for 55% of step latency, while at 72% skipped success falls significantly, to 51%; second, the success lost to visual reuse at the threshold of 0.05 is of similar size for the ten-step policy (19 points) and the one-step distilled policy (15 points), so no extra sensitivity of the one-step policy to visual staleness was found.

2. Related Work

VLA inference efficiency. VLA-Cache [4] reuses key-value entries associated with minimally changed visual tokens; EfficientVLA [5] prunes layers and visual tokens and caches intermediate features of the action head; VLA-Pruner [6] selects tokens by combined semantic and action relevance; DeeR-VLA [7] applies input-dependent early exit; subsequent variants continue these directions. These works report latency, throughput, or FLOPs, not energy. Concurrent work by Zhou et al. [15] characterizes OpenVLA, pi0, pi0.5, SmolVLA and GR00T across desktop GPUs, Jetson AGX Orin, Jetson Thor and NPUs with a physical power meter, and reports energy per inference alongside, but not divided by, LIBERO success. The present work differs in three respects. It targets the Orin Nano, an 8 GB module with a 15 W to 25 W envelope against 64 GB and up to 60 W for AGX Orin, on which SmolVLA does not run without the decomposition of Section 3. It reports energy per successful task, so that a configuration which saves energy by failing more often is charged for its failures. And it measures the energy-accuracy trade-off of individual inference optimizations rather than the platform-level cost of unmodified models. The visual-reuse mechanism examined here is deliberately the simplest member of this family, a per-camera mean absolute pixel difference threshold, because the object of study is the measured energy-accuracy relationship rather than the gating mechanism itself. Any of the cited mechanisms could be substituted within the same measurement harness.

Few-step action generation. Consistency Policy [8] and ManiCM [9] distill diffusion policies to one or two sampling steps. MeanFlow-based policies train one-step models directly [12]. SnapFlow [10] distills flow-matching VLAs of the pi0 family [14] to a single step. The present work contributes an account of the failure modes encountered when fine-tuning a converged action expert for distillation (Section 5.3), together with a low-rank recipe that avoids them and energy measurements for the resulting policy.

Embedded VLM energy. Prior work [11] measures the inference energy of vision-language chat models on Jetson-class hardware. That setting includes neither an action head nor a closed-loop task, and therefore no success denominator. The present work measures a complete perception-to-action policy against a manipulation benchmark.

3. Deployment Pipeline

Target platform. Jetson Orin Nano (8 GB, Ampere architecture, JetPack R36.4, TensorRT 10.3, power model 2). No PyTorch build exists for this configuration, and the available onnxruntime distribution provides only CPU execution; TensorRT is the sole GPU execution path.

Decomposition. SmolVLA inference comprises a prefix prefill, in which a SigLIP encoder and a 32-layer text transformer process 177 prefix tokens (two camera views of 64 tokens each, 48 language tokens, and one state token) to produce per-layer key-value tensors, followed by N=10 flow-matching denoising steps of a cross-attending action expert that emits a 50-action chunk. The deployed engines are: a per-camera vision engine (fp16); two prefill engines covering text-transformer layers 0-13 and 14-31 respectively (fp32; the split is required by builder memory limits and the precision by the analysis in Section 3.1); and a denoising engine executing one expert step (fp16), driven by a host-side Euler loop. The denoising engine is stateless with respect to the prefix: its key-value inputs are supplied afresh at each invocation, which removes the in-place cache mutation that otherwise prevents export. The exported chain reproduces eager execution exactly in fp32 and to a maximum action-space deviation of 2.5e-3 in the deployed mixed precision. Under the benchmark's evaluation protocol, in which one action is executed per replanning step, one chunk inference corresponds to one environment step.

3.1 Deployment failure modes

The following issues were encountered during pipeline construction; each either prevented an engine build or produced silently incorrect results.

1. Half-precision normalization overflow. The policy scales prefix embeddings by the square root of the hidden dimension (approximately 31), which drives layer-normalization variance accumulations beyond the fp16 representable range. TensorRT emits a warning but completes the build; the resulting prefill outputs are grossly incorrect, while the padded action dimensions of the final output remain near their reference values, which conceals the fault from cursory inspection. The prefill engines are therefore built in fp32; the vision and denoising engines are unaffected, with errors below 0.3%.
2. Builder memory limits. A monolithic prefill engine exceeds the compiler's constant-region allocation on an 8 GB device; splitting the text transformer at layer 14 yields two regions that build successfully. Weight stripping does not avoid build-time materialization.
3. The TensorRT 10.3 ONNX parser rejects ScatterND nodes that carry a reduction attribute, including the default value 'none'; the attribute is removed by graph transformation.
4. The PyTorch 2.9 dynamo exporter emits a Where node with a floating-point constant feeding Gather indices in the SigLIP position-embedding subgraph; the constant is retyped to int64.
5. Re-serializing a graph together with its external weight data duplicates tied weights by a factor of approximately 2.5, which starves the device-side build; all graph transformations are therefore applied without rewriting the original weight file.
6. Checkpoint variants of the same policy family differ in transformer depth (16 versus 32 layers), not only in input configuration; layer-split points must be derived from the loaded model rather than assumed.

4. Measurement Protocol

Power is sampled from the module's INA3221 instrumentation (VDD_IN input rail) at 10 ms intervals. For each configuration, 635 recorded observation steps, comprising three benchmark episodes with images, states, and language captured at the policy interface during rollouts, are replayed three times consecutively; mean power over the execution window is integrated to obtain gross energy per step, and idle power (6.9 W), measured immediately before each run, is subtracted to obtain net energy. Recorded observations are used in preference to synthetic inputs for two reasons: reuse mechanisms are meaningful only on realistic frame sequences, and energy itself is input-dependent. Baseline execution on random tensors was measured at 5.89 J per step against 4.31 J per step on recorded frames, indicating that synthetic-input baselines overstate cost by approximately 37%.

Task success is evaluated on LIBERO-Spatial under the lerobot 0.6.1 protocol, with ten tasks and ten episodes per task (n=100 per configuration); rollout videos are retained. Energy per successful task is computed as the product of mean episode length (156.5 steps for baseline episodes, obtained from per-episode video frame counts) and energy per step, divided by the success rate. The same 156.5 steps are applied to every configuration; episode lengths were not recomputed for the other configurations. This understates energy for configurations with lower success, because failed episodes run longer than successful ones (in the baseline, all 30 failures ran to the 280-step limit).

5. Optimizations

5.1 Threshold-gated visual reuse

For each camera at each control step, if the mean absolute pixel difference relative to the most recently encoded frame falls below a threshold τ, the cached 64-token encoding is reused and the vision engine is not invoked. The identical gating logic is applied in the deployed pipeline and in the simulator evaluation, with the cache cleared between episodes, so that the reported energy and success characterize the same policy behavior. In the benchmark workload, the static third-person camera admits reuse on 62-99.8% of steps depending on τ, and the wrist camera on 3-94%.

5.2 Exact incremental prefill

The policy's prefix attention mask is block-structured: image and language tokens attend only among themselves, while the state token attends to all tokens and no token attends to the state token. Consequently, when both cameras are gated, the key-value rows of the 176 image and language tokens are unchanged by construction, and the prefix update reduces to a single-token decoding of the new state token against the cached rows. This is an exact reformulation rather than an approximation; agreement with full prefill in fp32 is within 3e-5. One implementation detail is noteworthy: the model's standard forward pass routes cached invocations through its cross-attention path, which cannot process a prefix-stream token, and the update must therefore drive the per-layer self-attention computation directly. In deployment the incremental path executed on 634 of 635 replay steps.

5.3 One-step distillation under a low-rank constraint

Direct fine-tuning of the converged action expert for step reduction failed in every configuration attempted, with closed-loop success collapsing to between 0% and 22% within 5,000 updates. Three distinct causes were isolated: training in unnormalized action space, as the framework applies normalization in a preprocessing pipeline external to the policy; a loss dominated by 25 padded action dimensions whose regression targets are trivially predictable from the network input, and which the

reference implementation excludes from its loss; and self-referential consistency targets that degrade jointly with the student. After all three were corrected, full fine-tuning at a learning rate of 2e-5 with warmup still collapsed closed-loop behavior while the offline flow-matching loss remained close to that of the pretrained model (0.41 versus 0.30), indicating that offline loss does not predict closed-loop viability at this scale. The recipe that succeeded constrains adaptation to rank-32 low-rank updates [13] on the expert's attention and feed-forward projections (9.8M trainable parameters), holds all base weights fixed, and uses the frozen base model, integrated over four Euler substeps, as the distillation target; the loss is computed on the seven active action dimensions of the single-jump prediction. Training required 3.5 hours on a single desktop GPU. The one-step policy attains 61% success against the teacher's 70% (n=100). The adapted weights merge algebraically into the base model, and the merged model exports through the unmodified pipeline.

6. Results

6.1 Baseline

Quantity	Value
Latency per 50-action chunk, eager fp32, desktop-class GPU	390 ms
Latency per step, deployed pipeline, Orin Nano, recorded frames	232.9 ms
Energy per step, recorded frames	4.310 J gross, 2.713 J net
Success, LIBERO-Spatial (n=100)	70.0%
Energy per successful task	963 J (0.27 Wh)

The deployed pipeline on the embedded module executes faster than eager fp32 execution of the same policy on a desktop-class GPU, at a measured active module power of approximately 18.5 W. Latency and energy in this table come from the same replay of recorded frames; the same pipeline on random tensors measured 298.6 ms per chunk. At the measured rate, a 100 Wh battery corresponds to approximately 374 successful tasks of computation (360,000 J divided by 963 J).

6.2 Joint energy-accuracy results

Configurations are labeled R (visual reuse), I (exact incremental prefill), and D (one-step distillation). All success figures derive from 100-episode evaluations; energy figures are device measurements on replayed observations.

Configuration	Vision skip rate	Success	J/step gross (net)	ms/step	J per successful task
Baseline, 10-step	0%	70.0%	4.310 (2.713)	232.9	963
R, τ=0.01	17.2%	67.0%	4.008 (2.502)*	219.4	
R, τ=0.02	34.6%	68.0%	3.415 (2.057)*	192.4	786
R, τ=0.05	71.7%	51.0%	2.811 (1.643)*	165.2	863
R+I, τ=0.05	71.7%	51.0%	2.124 (1.178)*	136.0	652
D only	0%	61.0%	3.109 (2.050)	154.5	798
R+D, τ=0.05	73.3%	46.0%	1.630 (0.997)	89.3	
R+I+D, τ=0.05	82.4% (replay)	46.0%†	0.940 (0.528)	59.3	

Configuration	Vision skip rate	Success	J/step gross (net)	ms/step	J per successful task
R+I+D, τ=0.1	96.9% (replay)	n/a‡	0.680 (0.340)	47.6	
R, τ=0.1, any depth	93.5%	4.0%			

* Energy is measured on device replay, which skips more than the simulator at equal τ: the τ=0.02 row uses the replay at τ=0.02 (49.8% skip) and the τ=0.05 rows the replay at τ=0.05 (82.4% skip), so those energies are lower than the simulated policy would consume; the τ=0.01 row uses the replay at τ=0.005 (16.7% skip), matched to the simulator's 17.2%. Energy per successful task uses the baseline's 15,650 steps for every row (Section 4) and is therefore optimistic where success is lower. † Success of the R+D configuration; component I is exact and introduces no additional error. ‡ Outside the usable operating range established below; reported as an energy floor only.

The best configuration whose success is not significantly different from baseline is R at τ=0.02: energy per successful task falls from 963 J to about 786 J, an 18% reduction, at 68% success (two-proportion z=0.31 against 70%), with no training required. R+I at τ=0.02 was not measured, so the gain from the exact prefill at a usable skip rate is not known. R+I at τ=0.05 costs 2.124 J per step, about 652 J per successful task and 32% below baseline, but its success of 51% is a significant drop from 70% (z=2.75), so it is not a deployable setting; its energy per task is also understated, for the reasons given in Section 4 and in the table note. The R+I+D configuration at τ=0.05 reaches 0.94 J gross (0.53 J net) per control step at 59.3 ms, a 4.6-fold energy reduction, at the measured accuracy cost of the R+D combination. Visual reuse degrades with skip rate: success is 67% to 68% up to a 34.6% skip rate, 51% at 71.7% and 4% at 93.5%, so the usable operating range ends somewhere between roughly 35% and 72% skip; points between were not measured.

6.3 Observations

Redundancy of per-step visual encoding. Vision accounts for 55% of baseline step latency, yet about a third of encoder invocations (34.6% skipped) can be elided with no measurable cost to success (68% versus 70%); at 72% skipped, success falls to 51%. Unconditional per-step re-encoding, the default in current VLA runtimes, is a source of avoidable energy expenditure in this workload.

Step reduction and visual staleness. At τ=0.05 the gate costs the ten-step policy 19 percentage points of success (51% versus 70%, at a 71.7% skip rate) and the one-step policy 15 (46% versus 61%, at 73.3%). Relative to their own ungated success both retain about three quarters (73% and 75%), so these data do not show that the one-step policy is more sensitive to stale visual conditioning. An earlier version of this paper reported such an interaction on the basis of an incorrect 68% figure for the ten-step policy; that claim is withdrawn. The interaction was not tested at lower skip rates, and combinations of optimizations should still be evaluated jointly rather than inferred from individually reported results.

Divergence of offline and closed-loop metrics. A fine-tuned expert whose offline flow-matching loss was 35% above the pretrained reference was behaviorally nonviable, at 0% success, while its loss trajectory throughout training appeared unremarkable. Early closed-loop evaluation of intermediate checkpoints proved necessary for diagnosing training health.

6.4 The policy spectrum: where energy lives

The measurements above characterize one policy. To test whether the accounting generalizes, three further policies were deployed on the same device through the same TensorRT pipeline and measured

on the same power rail: DCE (0.2M parameters, a depth-conditioned drone navigation policy), ViNT (30M, a transformer visual navigation policy), and NoMaD (19M, a diffusion visual navigation policy running ten denoising steps per control step). Together with SmolVLA (450M) these span three orders of magnitude in parameter count.

Policy	Parameters	Gross mJ/step	Net mJ/step
DCE	0.2M	10.10	1.55
ViNT	30M	42.78	12.07
NoMaD	19M	218.59	69.21
SmolVLA	450M	4310	-

Gross energy includes the module's idle draw, measured at 6.90 W; net energy subtracts it. Both are reported because the distinction is large on a platform whose idle draw is comparable to its active draw, and because a comparison across platforms that does not state which convention it uses is uninterpretable.

The ordering is not monotonic in parameter count. NoMaD, at 19M parameters, consumes 5.1 times the energy of ViNT at 30M (5.7 times on net), because it executes ten sequential denoising passes per control step where ViNT executes one forward pass. Parameter count is therefore not a usable proxy for the energy cost of a policy at the edge; the number of sequential passes through the network is the dominant term.

A second measurement isolates the mechanism. The visual-reuse gate of Section 5.1 was applied to both navigation policies at a threshold that elides 99.5% of encoder invocations. This skip rate is far beyond the failure boundary established in Section 6.2, so the resulting configurations are not proposed as deployable; they instead measure the energy floor that each architecture approaches when its visual encoder is made arbitrarily cheap.

Policy	Baseline mJ/step	99.5% skip mJ/step	Reduction
ViNT	42.78	2.26	18.9x
NoMaD	218.59	141.09	1.55x

The identical intervention returns 18.9x on one policy and 1.55x on the other. ViNT approaches zero because its encoder constitutes nearly the whole network. NoMaD does not, because eliminating vision entirely leaves its ten denoising passes untouched, and those are where its energy resides. The consequence for practitioners is that the effectiveness of an inference optimization is predictable from the structure of the policy before the optimization is implemented: encoder elision is worthwhile in proportion to the encoder's share of the computation, and diffusion-dominated policies require step-count reduction instead.

7. Limitations

The joint energy-accuracy analysis covers one policy, one benchmark suite, one device, and one evaluation seed set, with n=100 per configuration corresponding to a standard error of approximately 4.6 percentage points near 70% success. Energy is measured on replayed observation streams, which are deterministic and comparable across configurations, rather than in closed loop on the device; the deployed implementation is numerically validated against the reference policy end to end. Gate

thresholds operate on raw pixel differences and were not tuned per camera; simulator and replay skip rates differ at equal τ (17-72% versus 33-82% for τ from 0.01 to 0.05) owing to preprocessing-domain differences, and results are reported as simulator-measured success paired with device-measured energy at the same τ, except that the τ=0.01 row uses the replay at τ=0.005, which matches the simulator skip rate. At τ=0.02 and τ=0.05 the replay skips more than the simulator, so the paired energy is lower than the simulated policy would consume. Energy per successful task applies the baseline step count to every configuration. The nine-point success gap of the distilled policy may narrow under distillation objectives developed concurrently with this work [10]; the present contribution is the energy accounting and the failure analysis rather than a claim regarding distillation quality. The four-policy comparison of Section 6.4 shares the single-device limitation and reports single runs per configuration; repeated measurement of the same configuration varied by 5 to 8%, so the reported ratios are robust to that variation but the absolute values should be read at that precision. Task success was not evaluated for the two navigation policies, so their energy figures are reported per control step rather than per successful task.

8. Reproducibility

Released materials comprise the engine decomposition and export tooling, including the graph transformations of Section 3.1; the on-device measurement harness; the gated simulator evaluation; the distillation trainer with early closed-loop checkpointing; and the per-episode accounting scripts. The hardware requirement is one Jetson Orin Nano and one CUDA-capable GPU; no training run in this work exceeded 3.5 hours on a single desktop GPU. The measurement harness is released under the name Joules Per Task (JPT).

Figures

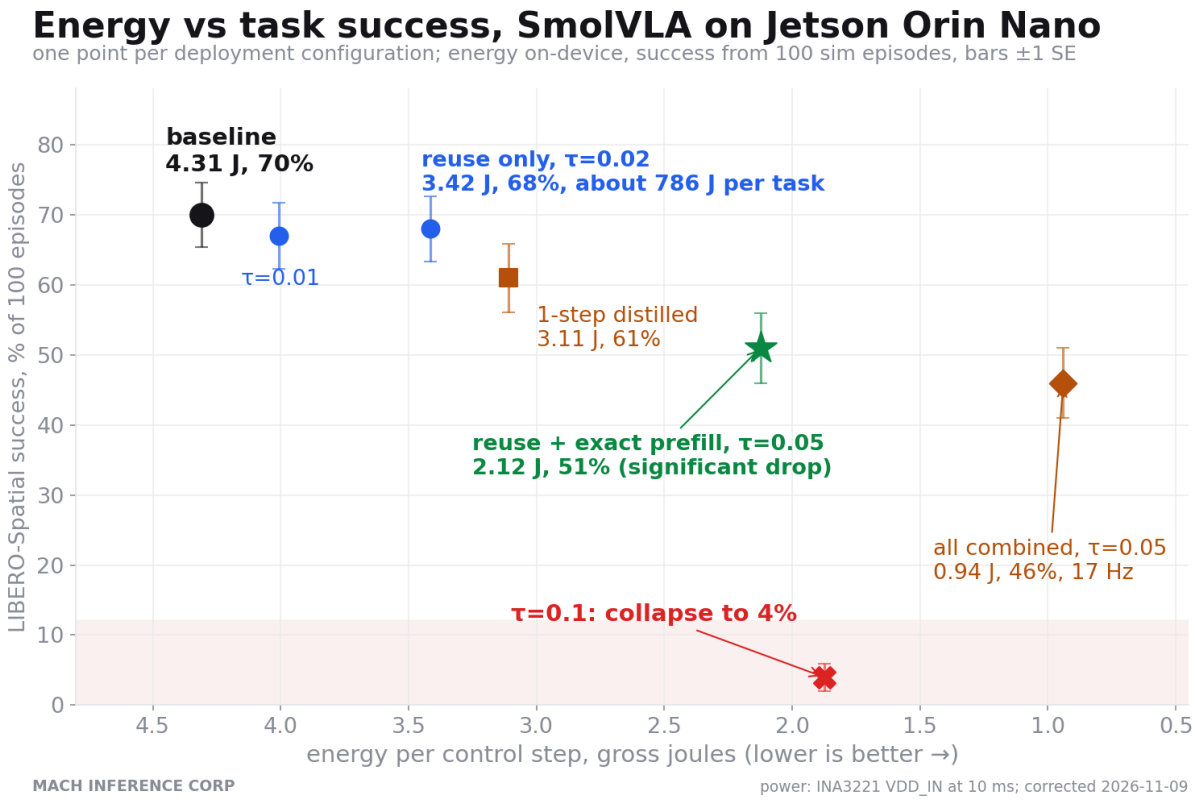


Figure 1. Energy-accuracy Pareto frontier (redrawn 2026-11-09 with the corrected 51% success at τ=0.05).

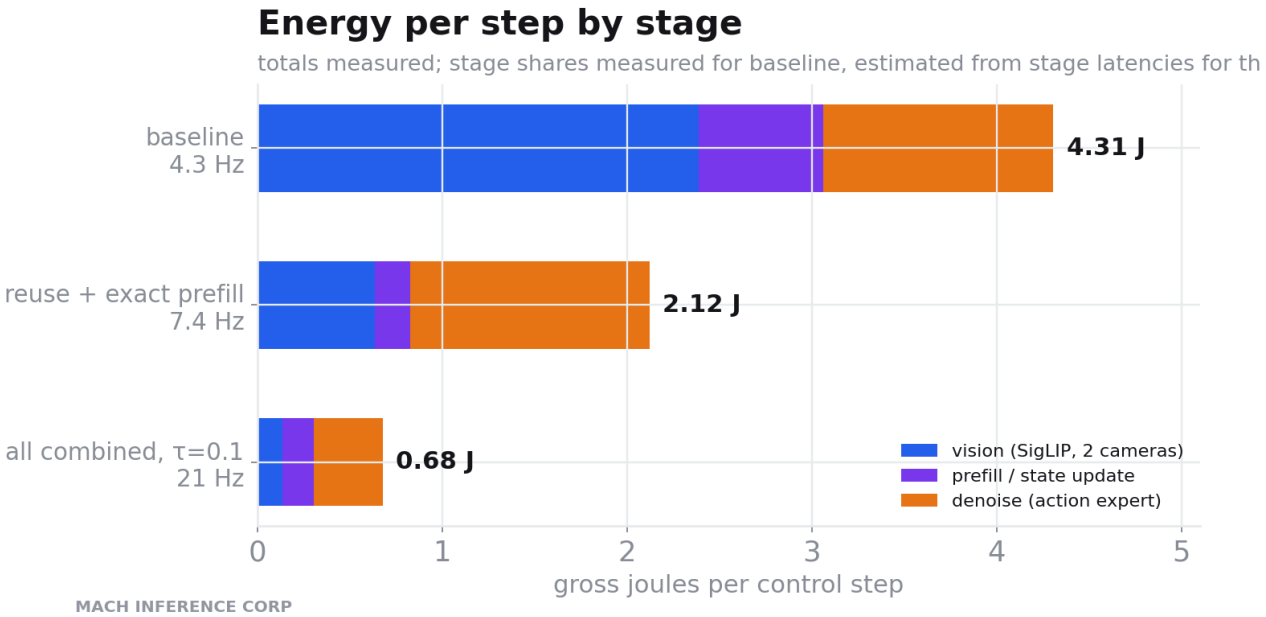


Figure 2. Per-stage energy breakdown.

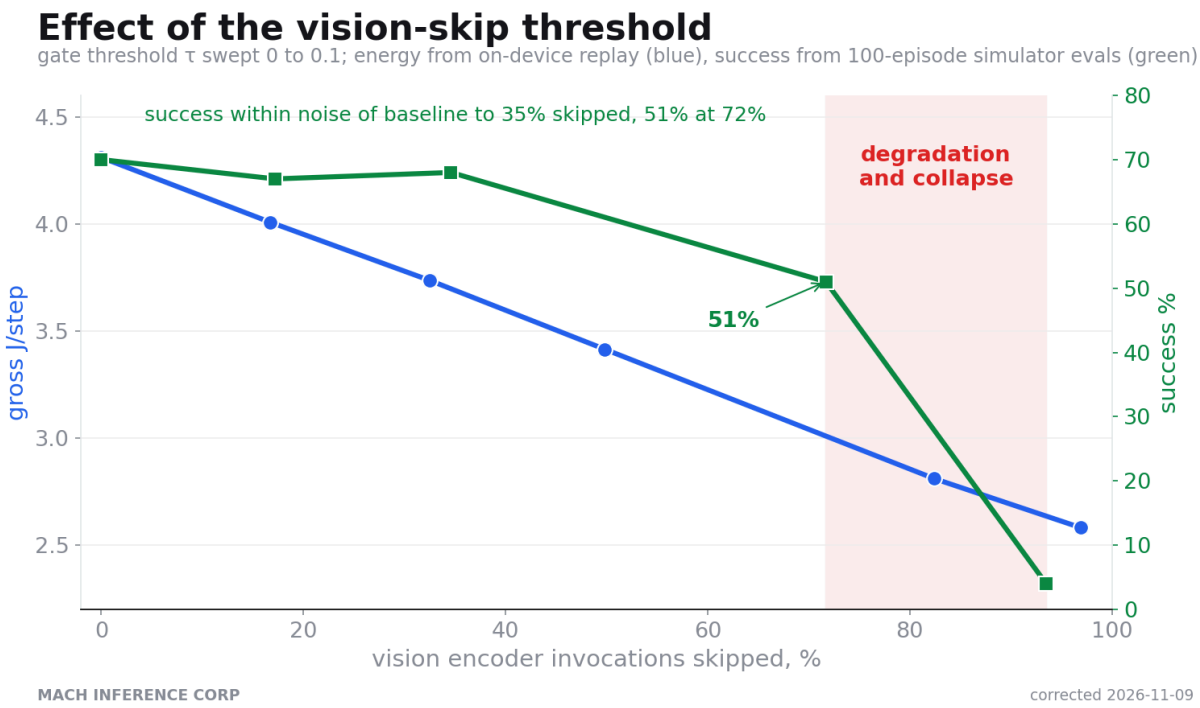


Figure 3. Skip rate against success, showing the failure boundary (redrawn 2026-11-09).

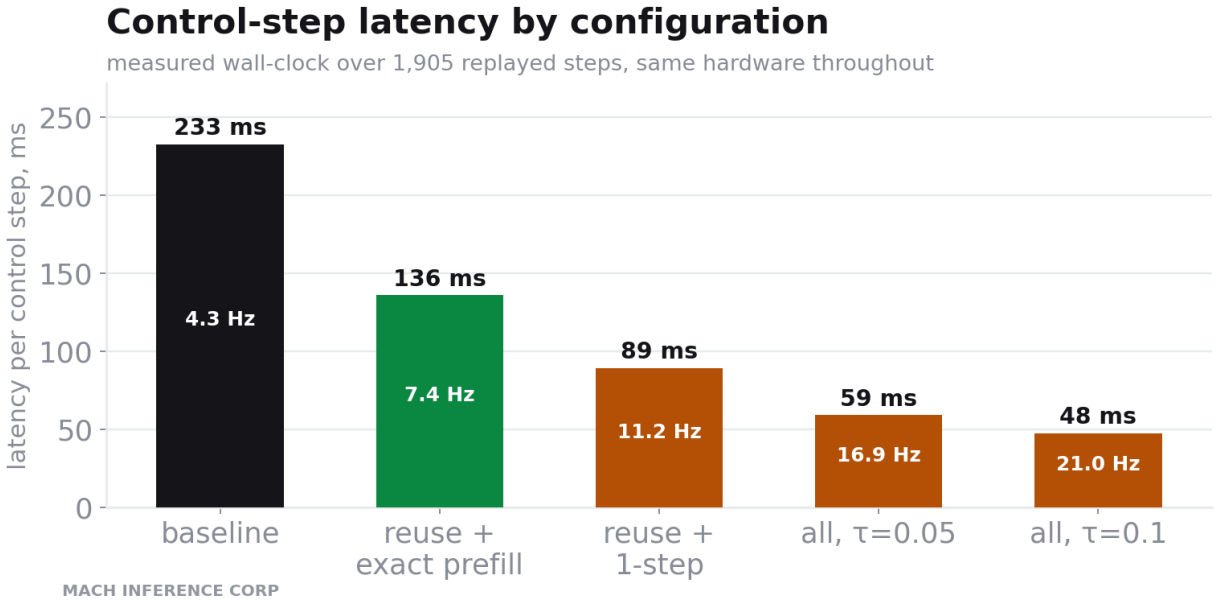


Figure 4. Latency by configuration.

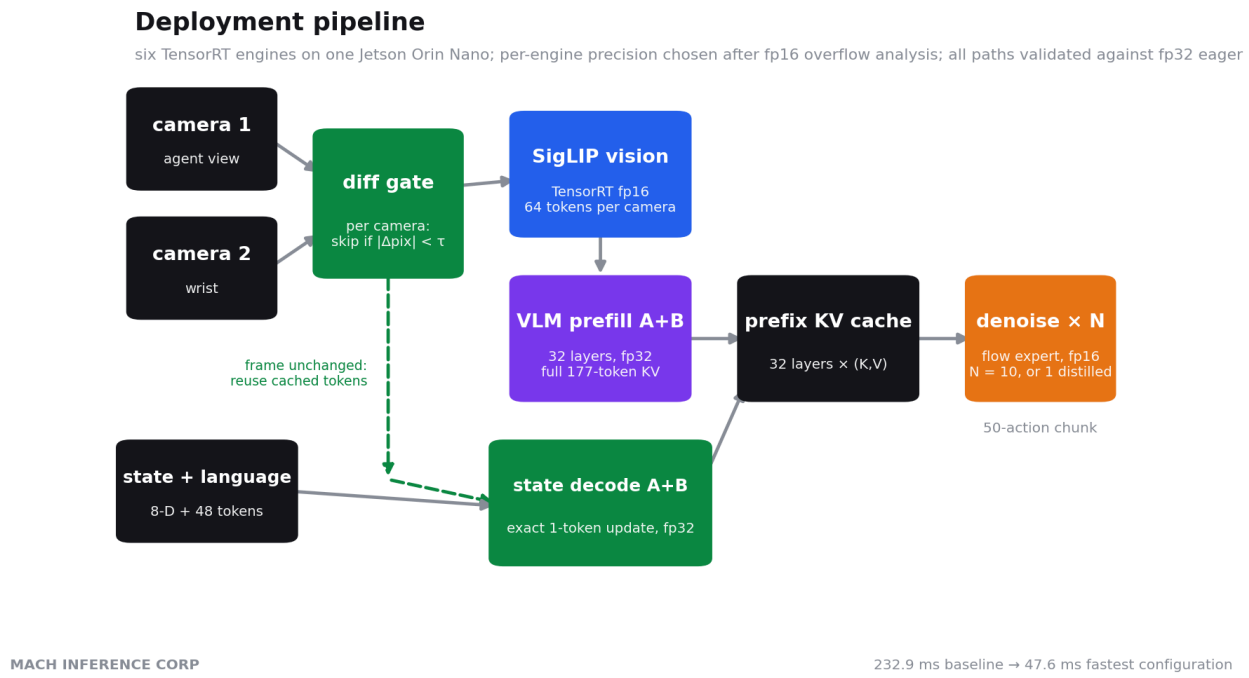


Figure 5. Multi-engine TensorRT decomposition.

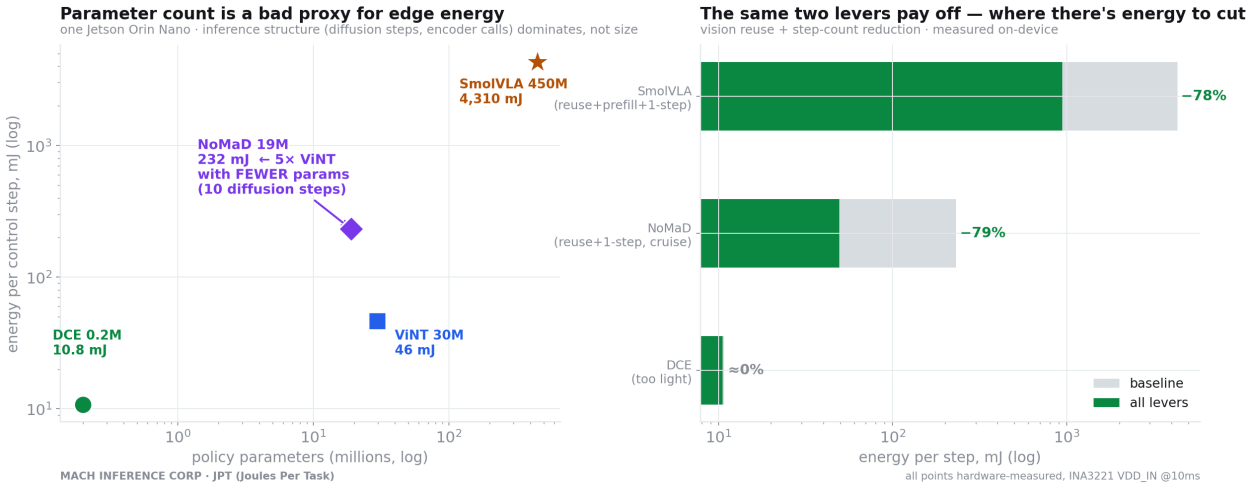


Figure 6. Four policies across three orders of magnitude of parameter count, and the payoff of the identical visual-reuse gate on two of them.

References

[1] Li et al. From Inference Efficiency to Embodied Efficiency: Revisiting Efficiency Metrics for VLA Models. arXiv:2603.19131, 2026.

[2] Shukor et al. SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics. arXiv:2506.01844, 2025.

[3] Liu et al. LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning. NeurIPS Datasets and Benchmarks, 2023.

[4] Hsu et al. VLA-Cache: Towards Efficient VLA via Adaptive Token Caching. arXiv:2502.02175, 2025.

[5] EfficientVLA: Training-Free Acceleration for VLA Models. arXiv:2506.10100, 2025.

[6] VLA-Pruner: Dual-Level Token Pruning for VLA Inference. arXiv:2511.16449, 2025.

[7] Yue et al. DeeR-VLA: Dynamic Early-Exit for Multimodal Robot LLMs. arXiv:2411.02359, 2024.

[8] Prasad et al. Consistency Policy: Accelerated Visuomotor Policies via Consistency Distillation. arXiv:2405.07503, 2024.

[9] ManiCM: Real-time 3D Diffusion Policy via Consistency Models. arXiv:2406.01586, 2024.

[10] SnapFlow: One-Step Action Generation for Flow-Matching VLAs via Progressive Self-Distillation. arXiv:2604.05656, 2026.

[11] Seeing is Free, Speaking is Not: The Energy Bottleneck in Edge VLM Inference. arXiv:2607.09520, 2026.

[12] Frans et al. One Step Diffusion via Shortcut Models. arXiv:2410.12557, ICLR 2025.

[13] Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685, 2021.

[14] Black et al. pi0: A Vision-Language-Action Flow Model for General Robot Control. arXiv:2410.24164, 2024.

[15] Zhou et al. Characterizing Vision-Language-Action Models across XPUs: Constraints and Acceleration for On-Robot Deployment. arXiv:2604.24447, 2026.